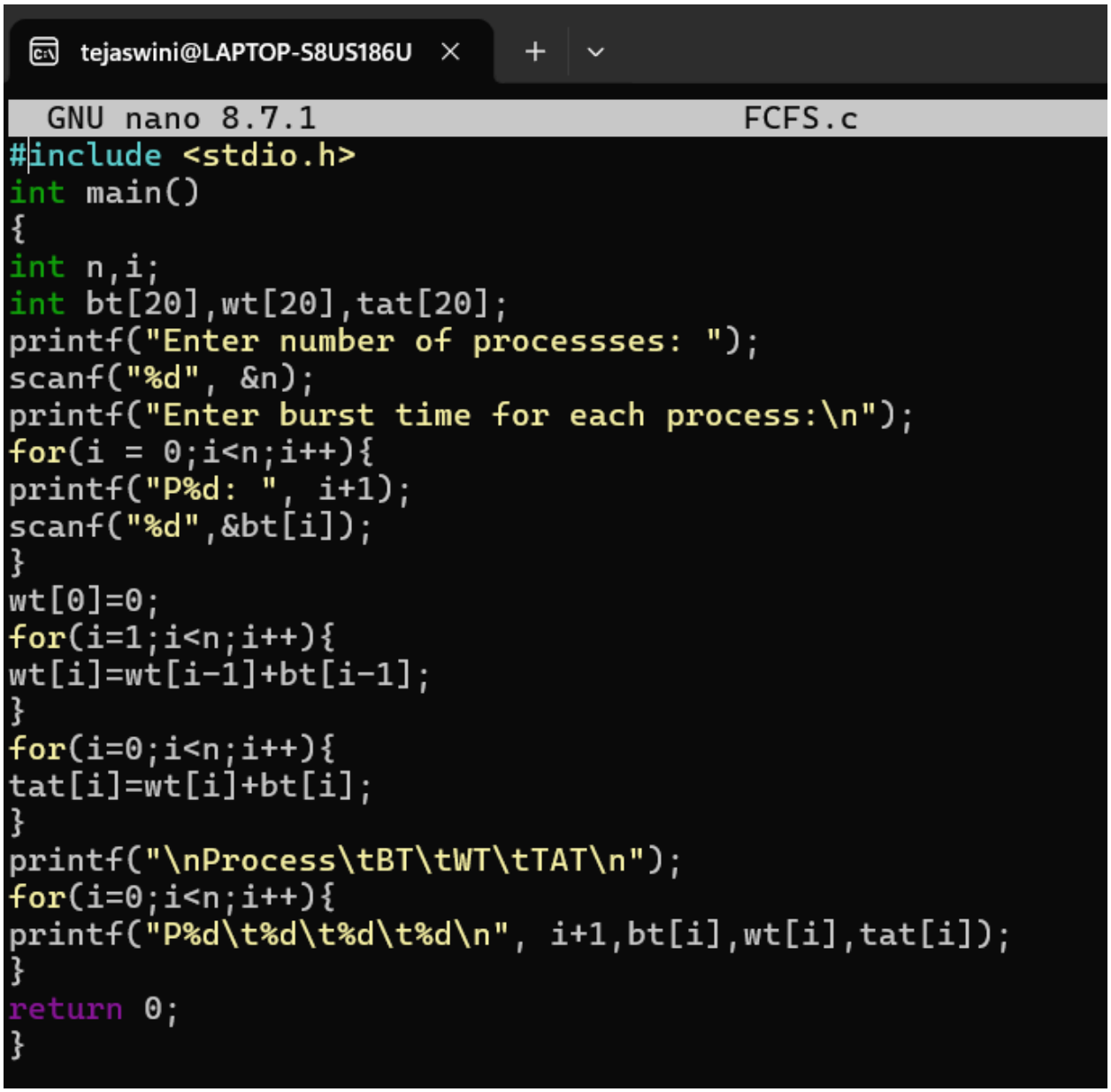
Scheduling_Algorithms

1.FCFS Algorithm



Output:

```
tejaswini@LAPTOP-S8US186U:~/OS_Lab$ gcc FCFS.c -o FCFS
tejaswini@LAPTOP-S8US186U:~/OS_Lab$ ./FCFS
Enter number of processses: 3
Enter burst time for each process:
P1: 6
P2: 9
P3: 8

Process BT      WT      TAT
P1      6       0       6
P2      9       6       15
P3      8       15      23
tejaswini@LAPTOP-S8US186U:~/OS_Lab$
```

2.SJF_non-preemptive

tejaswini@LAPTOP-S8US186U

GNU nano 8.7.1 SJF_non_prem.c *

```c
#include <stdio.h>

int main() {
    int n, i, j;
    int bt[20], wt[20], tat[20];
    int temp;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter burst time:\n");
    for (i = 0; i < n; i++) {
        printf("P%d: ", i + 1);
        scanf("%d", &bt[i]);
    }
    // Sort burst times
    for (i = 0; i < n - 1; i++) {
        for (j = i + 1; j < n; j++) {
            if (bt[i] > bt[j]) {
                temp = bt[i];
                bt[i] = bt[j];
                bt[j] = temp;
            }
        }
    }
    // Waiting time
    wt[0] = 0;

    for (i = 1; i < n; i++) {
        wt[i] = wt[i - 1] + bt[i - 1];
    }
    // Turnaround time
    for (i = 0; i < n; i++) {
   }
   printf("\nProcess\tBT\tWT\ttTAT\n");
   for (i = 0; i < n; i++) {
       printf("P%d\t%d\t%d\t%d\n",
              i + 1, bt[i], wt[i], tat[i]);
   }

   return 0;
}
```

Output:

```
tejaswini@LAPTOP-S8US186U:~/OS_Lab$ nano SJF_non_prem.c
tejaswini@LAPTOP-S8US186U:~/OS_Lab$ gcc SJF_non_prem.c -o SJF_non_prem
tejaswini@LAPTOP-S8US186U:~/OS_Lab$ ./SJF_non_prem
Enter number of processes: 3
Enter burst time:
P1: 9
P2: 4
P3: 2

Process BT      WT      tTAT
P1      2       0       2
P2      4       2       6
P3      9       6       15
tejaswini@LAPTOP-S8US186U:~/OS_Lab$
```

3.Priority

tejaswini@LAPTOP-S8US186U

GNU nano 8.7.1 priority.c *

```c
#include <stdio.h>

int main() {
    int n, i, j;
    int bt[20], priority[20], wt[20], tat[20];
    int temp;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter burst time and priority:\n");

    for (i = 0; i < n; i++) {
        printf("P%d Burst Time: ", i + 1);
        scanf("%d", &bt[i]);

        printf("P%d Priority: ", i + 1);
        scanf("%d", &priority[i]);
    }

    // Sort according to priority
    for (i = 0; i < n - 1; i++) {
        for (j = i + 1; j < n; j++) {

            if (priority[i] > priority[j]) {

                temp = priority[i];
                priority[i] = priority[j];
                priority[j] = temp;

                temp = bt[i];
                bt[i] = bt[j];
                bt[j] = temp;
            }
        }
    }

    // Waiting time
    wt[0] = 0;

    for (i = 1; i < n; i++) {
        wt[i] = wt[i - 1] + bt[i - 1];
    }

    // Turnaround time
    for (i = 0; i < n; i++) {
        tat[i] = wt[i] + bt[i];
    }

    printf("\nProcess\tBT\tPriority\tWT\tTAT\n");

    for (i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t\t%d\t%d\n",
               i + 1, bt[i], priority[i], wt[i], tat[i]);
    }

    return 0;
}
```

Output:

```
tejaswini@LAPTOP-S8US186U:~/OS_Lab$ nano priority.c
tejaswini@LAPTOP-S8US186U:~/OS_Lab$ gcc priority.c -o priority
tejaswini@LAPTOP-S8US186U:~/OS_Lab$ ./priority
Enter number of processes: 5
Enter burst time and priority:
P1 Burst Time: 5
P1 Priority: 6
P2 Burst Time: 8
P2 Priority: 3
P3 Burst Time: 9
P3 Priority: 1
P4 Burst Time: 2
P4 Priority: 7
P5 Burst Time: 3
P5 Priority: 5

Process BT      Priority        WT      TAT
P1      9       1               0       9
P2      8       3               9       17
P3      3       5               17      20
P4      5       6               20      25
P5      2       7               25      27
tejaswini@LAPTOP-S8US186U:~/OS_Lab$
```

4.Round robin

```
 GNU nano 8.7.1                                                round_robin.c *
#include <stdio.h>

int main() {
    int n, i;
    int bt[20], rem_bt[20];
    int wt[20], tat[20];
    int quantum;
    int time = 0;
    int done;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter burst time:\n");

    for (i = 0; i < n; i++) {
        printf("P%d: ", i + 1);
        scanf("%d", &bt[i]);

        rem_bt[i] = bt[i];
        wt[i] = 0;
    }

    printf("Enter time quantum: ");
    scanf("%d", &quantum);

    // Round Robin
    while (1) {

        done = 1;

        for (i = 0; i < n; i++) {

            if (rem_bt[i] > 0) {

                done = 0;

                if (rem_bt[i] > quantum) {
                    time = time + quantum;
                    rem_bt[i] = rem_bt[i] - quantum;
                }
                else {
                    time = time + rem_bt[i];

                    wt[i] = time - bt[i];

                    rem_bt[i] = 0;
                }
            }
        }

        if (done == 1)
            break;
    }

    // Turnaround time
    for (i = 0; i < n; i++) {
        tat[i] = wt[i] + bt[i];
    }

    printf("\nProcess\tBT\tWT\tTAT\n");

    for (i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\n",
               i + 1, bt[i], wt[i], tat[i]);
    }

    return 0;
}
```

Output:

```
tejaswini@LAPTOP-S8US186U:~/OS_Lab$ gcc round_robin.c -o round_robin
tejaswini@LAPTOP-S8US186U:~/OS_Lab$ ./round_robin
Enter number of processes: 4
Enter burst time:
P1: 2
P2: 6
P3: 4
P4: 7
Enter time quantum: 2

Process BT      WT      TAT
P1      2       0       2
P2      6       10      16
P3      4       8       12
P4      7       12      19
tejaswini@LAPTOP-S8US186U:~/OS_Lab$
```